Tribhuvan University

Faculty of Humanities & Social Sciences

OFFICE OF THE DEAN

System Analysis & Design

BCA — Semester 3 — 2019 — Answer Sheet

Subject Code: CACS203

Group A

Attempt ALL the questions. [10 × 1 = 10]

1. i) Information systems aimed at improving routine business activities?
a) Management Information System b) Decision Support System c) Transaction Processing Systems d) Executive Information System

ii) Project life cycle consists of?
a) System analysis and design b) Programming and debugging c) Formulation and planning various activities d) None of the above

iii) Most important feature of spiral model?
a) Quality management b) Version control c) Risk Management d) None of the above

iv) …includes existing system, proposed system, system flow charts, modular design?
a) Feasibility report b) System specification report c) Design Specification Report d) System proposal report

v) Phase where developers decide roadmap for project plan?
a) System Design b) System Analysis c) Coding d) Testing

vi) Approach where new system is tested in one part before implementing in others?
a) parallel b) direct c) phased d) pilot

vii) …extends software beyond its original functional requirements?
a) Corrective maintenance b) Perfective maintenance c) Adaptive maintenance d) Preventive maintenance

viii) Which normal form removes partial dependencies?
a) First Normal Form b) Second Normal Form c) Third Normal Form d) Boyce-Codd Normal Form

ix) In ER diagrams, double ovals denote?
a) Derived attributes b) Composite attributes c) Multi-value attributes d) Simple attributes

x) Testing beyond normal operational capacity is?
a) Integration testing b) Stress testing c) Performance testing d) Acceptance testing

[10 × 1 = 10]

Answer: Answers:

i) c) Transaction Processing Systems — TPS are information systems that collect, store, modify, and retrieve the routine day-to-day transactions of an organization (e.g., payroll, order entry, reservation systems). They are designed to improve the efficiency of routine business activities.

ii) c) Formulation and planning various activities — The project life cycle consists of phases that include formulation of the project idea, planning various activities, scheduling, execution, monitoring, and closure.

iii) c) Risk Management — The spiral model, proposed by Barry Boehm, emphasizes iterative development with a focus on risk analysis at each cycle. Risk management is its most important feature, making it suitable for large, complex, and high-risk projects.

iv) c) Design Specification Report — The Design Specification Report documents the existing system analysis, proposed system details, system flowcharts, modular design, data dictionary, input/output designs, and database design.

v) b) System Analysis — In the System Analysis phase, analysts study the existing system, identify problems, gather requirements, and decide the roadmap and scope for the project plan, including feasibility studies.

vi) d) pilot — In the pilot approach (pilot conversion), the new system is implemented in one part of the organization (e.g., one branch or department) as a test before rolling it out to the entire organization.

vii) b) Perfective maintenance — Perfective maintenance involves enhancing and extending the software beyond its original functional requirements to improve performance, add new features, or refine existing functionality based on user feedback.

viii) b) Second Normal Form — Second Normal Form (2NF) removes partial dependencies, where a non-prime attribute is functionally dependent on part of a composite primary key. A relation is in 2NF if it is in 1NF and has no partial dependencies.

ix) c) Multi-value attributes — In ER diagrams, double ovals (or ellipses) represent multi-valued attributes, which are attributes that can hold multiple values for a single entity instance (e.g., a person can have multiple phone numbers).

x) b) Stress testing — Stress testing evaluates the system's stability and reliability under extreme conditions by pushing it beyond its normal operational capacity (e.g., heavy load, limited resources) to identify breaking points.

Group B

Attempt any SIX questions. [6 × 5 = 30]

3. When would you use agile methodologies? How is it different from waterfall approach?

[5]

Answer: When to use Agile Methodologies:
Agile methodologies are best suited for projects where:
• Requirements are uncertain, evolving, or likely to change frequently.
• The project is complex and requires frequent feedback from stakeholders.
• Rapid delivery of working software in small increments is preferred.
• The team is co-located and can collaborate closely with customers.
• There is a need for continuous improvement and adaptation.
• Startups, product development, and customer-facing applications benefit greatly from agile.

Differences between Agile and Waterfall Approach:
BasisAgileWaterfall Model typeIterative and incrementalLinear and sequential RequirementsEvolving and flexible; changes are welcomedFixed at the beginning; changes are costly DeliveryWorking software delivered frequently in sprintsSoftware delivered at the end of the project Customer involvementHigh; continuous collaboration with stakeholdersLow; customer involved mainly at start and end TestingContinuous testing throughout developmentTesting occurs only after development is complete DocumentationLightweight; working software over documentationHeavy documentation at each phase Risk managementRisks identified and mitigated in each iterationRisks may surface late in the project ExamplesScrum, Kanban, Extreme Programming (XP)Traditional SDLC model

4. Why is project management important? Explain activities performed by project manager during execution.

[5]

Answer: Importance of Project Management:
Project management is crucial because it:
• Ensures projects are completed on time, within budget, and to the required quality standards.
• Provides clear goals, scope, and direction, minimizing confusion and rework.
• Helps identify and mitigate risks before they become critical issues.
• Optimizes resource allocation (human, financial, technical) efficiently.
• Improves communication and coordination among team members and stakeholders.
• Facilitates tracking progress through milestones and deliverables.
• Increases the probability of project success and customer satisfaction.

Activities Performed by Project Manager During Execution:
1. Team Management: Assigning tasks to team members based on skills, motivating the team, resolving conflicts, and ensuring collaboration.

2. Monitoring Progress: Tracking project milestones, deadlines, and deliverables against the project plan using tools like Gantt charts and burn-down charts.

3. Quality Assurance: Ensuring that outputs meet the defined quality standards through reviews, inspections, and testing.

4. Risk Management: Continuously identifying new risks, assessing their impact, and implementing mitigation strategies.

5. Communication: Conducting regular status meetings, preparing progress reports, and maintaining communication with stakeholders, clients, and upper management.

6. Change Control: Managing scope creep by evaluating change requests, approving or rejecting changes, and updating plans accordingly.

7. Resource Allocation: Ensuring that necessary resources (hardware, software, personnel) are available when needed and reallocating as priorities shift.

8. Issue Resolution: Addressing technical and non-technical problems that arise during development promptly to avoid delays.

9. Budget Control: Monitoring expenditures, comparing actual costs against the budget, and taking corrective action if variances occur.

5. List various methods of interacting with a system. Explain factors to consider while designing a form.

[5]

Answer: Methods of Interacting with a System:

1. **Command-Line Interface (CLI):** Users interact by typing text commands into a terminal or console.
2. **Graphical User Interface (GUI):** Users interact through visual elements like windows, icons, menus, buttons, and pointers (WIMP).
3. **Menu-Driven Interface:** Users select options from a list of displayed menus.
4. **Form-Based Interface:** Users enter data into predefined fields on a form layout.
5. **Natural Language Interface:** Users communicate using everyday language (e.g., chatbots, voice assistants).
6. **Touch-Based Interface:** Users interact by touching the screen (common in mobile devices and kiosks).
7. **Voice User Interface (VUI):** Users give spoken commands (e.g., Siri, Alexa, Google Assistant).
8. **Gesture-Based Interface:** Users interact through physical movements detected by sensors or cameras.

Factors to Consider While Designing a Form:

1. Clarity and Simplicity: The form should be easy to understand with clear labels and logical flow. Avoid clutter and unnecessary fields.

2. Consistency: Use consistent fonts, colors, alignment, and spacing throughout the form. Buttons and input fields should follow a uniform style.

3. Logical Grouping: Related fields should be grouped together (e.g., personal information, address details) using visual separators or fieldsets.

4. Input Validation: Provide real-time validation and clear error messages to prevent incorrect data entry (e.g., date formats, email patterns).

5. Accessibility: Ensure the form is usable by people with disabilities by supporting keyboard navigation, screen readers, and sufficient color contrast.

6. Appropriate Field Types: Use the right input controls for the data type — dropdowns for limited options, radio buttons for single selection, checkboxes for multiple selections, and text areas for long text.

7. Defaults and Autofill: Pre-fill known data where possible and set sensible defaults to speed up data entry.

8. Navigation and Instructions: Provide clear instructions, tooltips, or help text. Include navigation buttons (Next, Previous, Submit, Cancel) at intuitive locations.

9. Minimize User Effort: Reduce the number of fields to only what is essential. Use auto-complete and smart defaults to minimize typing.

10. Feedback: Provide confirmation messages after submission and visual cues (loading indicators) during processing.

6. What are deliverables from coding and testing? Explain different approaches to installation.

[5]

Answer: Deliverables from Coding:

• **Source Code:** The actual program files written in the chosen programming language.
• **Compiled/Executable Code:** Machine-readable binaries or bytecode ready for deployment.
• **Code Documentation:** Inline comments, README files, and API documentation explaining the code structure.
• **Unit Test Results:** Reports showing that individual modules have been tested and passed.
• **Version Control Logs:** Commit history and change logs from tools like Git.

Deliverables from Testing:

• **Test Plans and Test Cases:** Documents detailing what was tested, how, and expected outcomes.
• **Test Scripts:** Automated test scripts for regression and continuous testing.
• **Bug/Defect Reports:** Documented list of identified bugs with severity, steps to reproduce, and status.
• **Test Summary Report:** Overall results including pass/fail statistics, coverage metrics, and risk assessment.
• **User Acceptance Test (UAT) Sign-off:** Formal approval from users/stakeholders confirming the system meets requirements.

Different Approaches to Installation:

1. Direct Cutover (Big Bang): The old system is completely replaced by the new system at a single point in time. It is simple but risky because any failure affects the entire organization immediately.

2. Parallel Installation: Both old and new systems run simultaneously for a period. Output is compared for accuracy before fully switching over. It is the safest but most expensive approach due to duplicate effort.

3. Phased Installation: The new system is implemented in stages or modules over time (e.g., by department or function). It reduces risk and allows learning from early phases, but integration between old and new parts can be complex.

4. Pilot Installation: The new system is introduced in one part of the organization (e.g., one branch or location) first. After successful evaluation, it is rolled out to other parts. It allows testing in a real environment with limited risk.

7. Why is normalization required? State second normal form and explain with example.

[5]

Answer: Why Normalization is Required:
Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. It is required because:
• **Eliminates Data Redundancy:** Prevents the same data from being stored in multiple places, saving storage space.
• **Prevents Update Anomalies:** Avoids insertion, deletion, and modification anomalies that occur when redundant data is updated inconsistently.
• **Ensures Data Integrity:** Maintains accuracy and consistency of data across the database.
• **Improves Query Performance:** Well-structured tables can be queried more efficiently.
• **Simplifies Maintenance:** Changes to data structure are easier to manage in normalized databases.

Second Normal Form (2NF):
A relation is in Second Normal Form (2NF) if:
1. It is already in First Normal Form (1NF).
2. It has no partial dependency — that is, no non-prime attribute is functionally dependent on a proper subset of any candidate key (i.e., no non-prime attribute depends on part of a composite key).

Example:
Consider a relation **Enrollment(StudentID, CourseID, StudentName, CourseName, Grade)** with composite primary key (StudentID, CourseID).

Functional Dependencies:
• StudentID → StudentName (partial dependency — StudentName depends on only part of the key)
• CourseID → CourseName (partial dependency — CourseName depends on only part of the key)
• (StudentID, CourseID) → Grade (full dependency)

This relation is in 1NF but NOT in 2NF because of partial dependencies.

Decomposition into 2NF:
Student (StudentID, StudentName)PK: StudentID **Course** (CourseID, CourseName)PK: CourseID **Enrollment** (StudentID, CourseID, Grade)PK: (StudentID, CourseID), FKs: StudentID, CourseID
Now all non-prime attributes are fully functionally dependent on the entire primary key in each relation. Thus, the database is in 2NF.

8. Construct an E-R Diagram for football club that has a name and a ground and is made up of players. A player can play for only one club and a manager identified by his name manages a club. A footballer has a registration number, name and age. A club manager also buys players. Each club plays against other clubs in the league and matches have a date, venue and score.

[5]

Answer: ER Diagram Description for Football Club:

Entities and Attributes:
EntityAttributesKey ClubName, GroundName (Primary Key) PlayerRegistrationNumber, Name, AgeRegistrationNumber (Primary Key) ManagerNameName (Primary Key) MatchDate, Venue, ScoreComposite key (Date, Venue) or MatchID
Relationships:
RelationshipBetween EntitiesCardinalityDescription Made_up_ofClub — Player1 : NA club is made up of many players; a player plays for only one club. ManagesManager — Club1 : 1A manager manages exactly one club; each club has exactly one manager. BuysManager — PlayerM : NA club manager buys many players; a player can be bought by managers over time (or interpret as Club buys Player). Plays_againstClub — Match — ClubM : N (recursive)Each club plays against other clubs in matches. A match involves two clubs. This is a recursive/ternary relationship.
Detailed ER Diagram Structure:
[Club] [Manager] | Name (PK) | Name (PK) | Ground | | | | 1 | 1 | | Made_up_of Manages | | | N | 1 | | [Player] [Club] | RegistrationNo (PK) | | Name | | Age | | | | M | +----------Buys----------->+ (Manager buys Player) [Match] is connected to two Club entities via Plays_against: Club (1) ----< Plays_against >---- Match Club (1) ----< (Date, Venue, Score) >----
Notes:
• Club and Player have a **one-to-many (1:N)** relationship because one club has many players, but each player belongs to only one club.
• Manager and Club have a **one-to-one (1:1)** relationship since one manager manages one club and vice versa.
• The "buys" relationship can be modeled between Manager and Player (M:N) or Club and Player depending on interpretation.
• The "plays against" relationship involves two clubs and a match; it can be represented as a ternary relationship or as a binary many-to-many between Club and Match with role names (HomeClub, AwayClub).
• In the diagram, rectangles represent entities, ovals represent attributes, diamonds represent relationships, and lines with cardinality ratios connect them.

9. Maintenance is an on-going process. Do you agree? Explain the process of maintaining information systems.

[5]

Answer: Yes, Maintenance is an On-going Process:
I completely agree that maintenance is an ongoing process. Information systems operate in dynamic environments where business requirements, technology, regulations, and user needs continuously change. Software does not wear out like physical hardware, but it becomes obsolete or defective over time due to changing conditions. Maintenance ensures that the system remains useful, efficient, secure, and aligned with organizational goals throughout its lifecycle. Studies show that maintenance can consume 60–80% of the total software lifecycle cost, highlighting its continuous nature.

Process of Maintaining Information Systems:
1. Problem Identification / Modification Request:
Users, developers, or managers identify issues, bugs, or enhancement needs and submit a formal change request or problem report.

2. Problem Analysis:
The maintenance team analyzes the request to understand its nature (bug fix, enhancement, adaptation), impact on the system, feasibility, and cost. A decision is made on whether to approve the request.

3. Design of Modification:
If approved, the team designs the changes needed. This may involve modifying code, database schemas, interfaces, or documentation. Impact analysis ensures that changes do not break existing functionality.

4. Implementation / Coding:
The actual code changes are implemented by developers following coding standards. New modules may be added or existing ones modified.

5. Testing:
Rigorous testing is performed including unit testing, integration testing, regression testing, and system testing to ensure the modification works correctly and has not introduced new errors.

6. Deployment:
The updated system is deployed to the production environment. This may involve patching, version releases, or rolling updates.

7. Documentation Update:
All related documentation (user manuals, system manuals, design documents) is updated to reflect the changes.

8. Review and Feedback:
Post-implementation review evaluates whether the maintenance objective was achieved. User feedback is collected for future improvements.

Types of Maintenance:
- **Corrective:** Fixing bugs and errors.
- **Adaptive:** Modifying the system to work in a changed environment (OS, hardware, regulations).
- **Perfective:** Enhancing performance or adding new features.
- **Preventive:** Making changes to prevent future problems.

Group C

Attempt any TWO questions. [2 × 10 = 20]

10. Develop a context diagram and top level logical DFD for B&B; mail-order company that distributes CDs, DVDs of music, games, movies, software at discount prices to club members. When an order processing clerk receives an order form, he/she verifies that the sender is a club member by checking the Member file. If not a member, returns the order with a membership application form. If a member, verifies order item data by checking the Item file. Then enters order data and saves it to the Daily Order file. Also prints an invoice and shipping list for each order, forwarded to Order Fulfillment Department.

[10]

Answer: Context Diagram (Level 0 DFD) for B&B; Mail-Order Company:
A Context Diagram shows the entire system as a single process with external entities and data flows.
External Entities: Data Flows: [Club Member] --------Order Form----------> [0] B&B; Mail-Order System [Club Member] <--------Membership App Form-- (if not member) [Club Member] <--------Invoice---------- (if member) [Club Member] <--------Shipping List-------- (if member) [Order Fulfillment Dept] <--Shipping List--- (if member)
Description of Context Diagram:
• **External Entity: Club Member** — sends Order Form to the system; receives Membership Application Form (if not a member), Invoice, and Shipping List (if a member).
• **External Entity: Order Fulfillment Department** — receives the Shipping List to process physical delivery.
• **Process: 0 B&B; Mail-Order System** — the entire system represented as a single bubble.
• **Data Stores (implied but not shown in context diagram):** Member File, Item File, Daily Order File.

Top Level Logical DFD (Level 1 DFD) for B&B; Mail-Order Company:
The Level 1 DFD decomposes the single process into sub-processes:
[Club Member] ---Order Form---> [1.0] Receive & Verify Order | | Check Member Status v [D1] Member File | +-----------------+-----------------+ | Not Member | Member v v [Club Member] <--Membership App Form-- [2.0] Verify Order Items | | Check Item Data v [D2] Item File | v [3.0] Process Order | +------------+------------+ | | v v [D3] Daily Order File [4.0] Print Documents | +------------------------+------------------------+ | | v v [Club Member] <---Invoice--- [Order Fulfillment Dept] <---Shipping List---
Description of Level 1 DFD Processes:
Process 1.0 — Receive & Verify Order:
The order processing clerk receives the Order Form and verifies whether the sender is a club member by querying the **Member File (D1)**.
• If **not a member**: returns the Order Form along with a **Membership Application Form** to the Club Member.
• If **a member**: passes the order to Process 2.0.

Process 2.0 — Verify Order Items:
The clerk verifies the ordered items (CDs, DVDs, games, movies, software) by checking the **Item File (D2)** to ensure item data is correct and available.

Process 3.0 — Process Order:
The clerk enters the verified order data and saves it to the **Daily Order File (D3)**. This creates a permanent record of the transaction.

Process 4.0 — Print Documents:
The system generates and prints two documents:
• **Invoice** — sent to the Club Member for payment.
• **Shipping List** — sent to the Order Fulfillment Department to pack and ship the items.

Data Stores:
• **D1 — Member File:** Contains member details used to verify club membership.
• **D2 — Item File:** Contains catalog information about CDs, DVDs, games, movies, and software.

• **D3 — Daily Order File:** Accumulates all orders processed during the day.

Data Flows Summary:
Data FlowSourceDestination Order FormClub MemberProcess 1.0 Member StatusD1 Member FileProcess 1.0 Membership Application FormProcess 1.0Club Member Verified OrderProcess 1.0Process 2.0 Item DataD2 Item FileProcess 2.0 Validated OrderProcess 2.0Process 3.0 Order RecordProcess 3.0D3 Daily Order File Print RequestProcess 3.0Process 4.0 InvoiceProcess 4.0Club Member Shipping ListProcess 4.0Order Fulfillment Dept

11. With proper reasoning, explain how CASE Tools aid in information system development? You have been hired as a system analyst in TU tech software development company and are asked to analyze the way system works. What qualities do you need to have to analyze such type of systems?

[10]

Answer: How CASE Tools Aid in Information System Development:
CASE (Computer-Aided Software Engineering) Tools are software applications that automate and support various activities throughout the SDLC. They significantly improve productivity, quality, and consistency in information system development.

1. Requirement Analysis and Modeling:
CASE tools help analysts create visual models such as Data Flow Diagrams (DFDs), Entity-Relationship (ER) diagrams, UML diagrams (use case, class, sequence), and process models. Tools like Visio, Enterprise Architect, and Rational Rose provide drag-and-drop interfaces that make modeling faster and error-free. This ensures requirements are clearly documented and communicated.

2. Documentation Generation:
CASE tools automatically generate technical documentation, user manuals, and data dictionaries from models and code. This saves significant manual effort and ensures documentation stays synchronized with the system design.

3. Code Generation:
Upper CASE and Integrated CASE (I-CASE) tools can automatically generate code skeletons, database schemas (SQL), and even complete application code from design specifications. This reduces coding time and human error.

4. Project Management Support:
Many CASE tools include features for scheduling, resource allocation, progress tracking, and version control. They help managers monitor milestones, identify bottlenecks, and ensure timely delivery.

5. Reverse Engineering:
CASE tools can analyze existing source code and generate design diagrams and documentation. This is invaluable for maintaining legacy systems or understanding undocumented code.

6. Testing and Quality Assurance:
Some CASE tools support test case generation, simulation of models, and consistency checking between design and code. They help identify defects early in the lifecycle when they are cheaper to fix.

7. Repository Management:
Centralized repositories store all project artifacts (models, code, documents) in one place, enabling team collaboration, version control, and reuse of components across projects.

8. Standardization:
CASE tools enforce methodological standards (structured or object-oriented), ensuring that all team members follow consistent notations and practices.

Reasoning / Conclusion:
By automating repetitive tasks, reducing manual errors, improving communication through visual models, and maintaining consistency across phases, CASE tools dramatically increase development speed, reduce costs, and improve software quality.

Qualities Needed to Analyze Systems as a System Analyst at TU Tech:

1. Analytical Thinking: Ability to break down complex systems into components, identify patterns, understand interdependencies, and diagnose problems logically.

2. Technical Knowledge: Strong understanding of databases, networks, programming concepts, software architecture, and current technologies to comprehend how systems function technically.

3. Communication Skills: Must interact effectively with technical teams, non-technical users, and management. Needs to conduct interviews, write clear reports, and present findings persuasively.

4. Problem-Solving Ability: Capacity to identify bottlenecks, inefficiencies, and risks in existing systems and propose practical, cost-effective solutions.

5. Attention to Detail: Must meticulously document requirements, data flows, and processes without overlooking edge cases or exceptions.

6. Business Domain Knowledge: Understanding of the company's business processes, goals, and industry trends to align technical solutions with business needs.

7. Interpersonal and Negotiation Skills: Ability to manage stakeholder expectations, resolve conflicts between user groups, and build consensus on system changes.

8. Adaptability and Learning Agility: Technology changes rapidly; an analyst must be willing to learn new tools, methodologies, and domains quickly.

9. Documentation and Modeling Skills: Proficiency in creating DFDs, ERDs, UML diagrams, process maps, and writing specifications using CASE tools and diagramming software.

10. Ethics and Integrity: Must handle sensitive organizational data responsibly and recommend solutions that are in the best interest of the organization rather than personal gain.

11a. Why do software projects often fail? Explain different types of software testing.

[10]

Answer: Why Software Projects Often Fail:
Software project failure is a common phenomenon in the IT industry. The major reasons include:

1. Unclear or Changing Requirements:
If requirements are poorly defined, ambiguous, or constantly changing without proper change control, the final product may not meet user expectations. Scope creep is a major contributor to failure.

2. Poor Planning and Estimation:
Unrealistic deadlines, underestimated budgets, and inadequate resource allocation lead to rushed development, cutting corners, and compromised quality.

3. Inadequate Communication:
Lack of communication between stakeholders, developers, and users results in misunderstandings, misaligned goals, and missed expectations.

4. Lack of User Involvement:
When end users are not involved throughout the development process, the system may not address real-world needs, leading to rejection after deployment.

5. Poor Project Management:
Ineffective leadership, lack of risk management, insufficient tracking of progress, and inability to resolve issues promptly can derail projects.

6. Technical Complexity:
Overly ambitious architectures, use of immature technologies, poor code quality, and inadequate testing create insurmountable technical debt.

7. Insufficient Testing:
When testing is treated as an afterthought or given insufficient time, critical bugs reach production, damaging user trust and requiring expensive fixes.

8. Staff Turnover and Skill Gaps:
Losing key team members or hiring underqualified personnel disrupts momentum and reduces team capability.

9. Organizational Issues:
Political conflicts, lack of executive support, competing priorities, and resistance to change within the organization can sabotage even well-planned projects.

Different Types of Software Testing:
TypeDescription Unit Testing Tests individual modules or functions in isolation to verify they work correctly. Usually done by developers. **Integration Testing**Tests the interaction between integrated modules to detect interface defects and data flow issues. **System Testing**Tests the complete, integrated system as a whole to verify it meets specified requirements. **Acceptance Testing**Conducted by end users or clients to determine whether the system is acceptable for delivery (UAT — User Acceptance Testing). **Regression Testing**Re-tests the system after modifications to ensure that existing functionality still works and new bugs were not introduced. **Alpha Testing**Internal testing performed by the organization before releasing the

software to external users. **Beta Testing**External testing by a limited number of real users in their own environment to identify issues before general release. **Performance Testing**Evaluates system responsiveness, throughput, and stability under expected load conditions. **Stress Testing**Pushes the system beyond normal operational limits to identify its breaking point and ensure graceful failure. **Security Testing**Identifies vulnerabilities, threats, and risks in the system to protect data and prevent unauthorized access. **Usability Testing**Evaluates how easy and intuitive the system is for end users to learn and operate. **White-Box Testing**Tests internal structure, logic, and code paths with knowledge of the source code. **Black-Box Testing**Tests functionality without knowledge of internal code, based solely on requirements and inputs/outputs.

11b. List features of OOAD. Differentiate between structured methodologies and object oriented methodologies.

[10]

Answer: Features of OOAD (Object-Oriented Analysis and Design):
OOAD is an approach to software engineering that models a system as a group of interacting objects. Its key features include:

1. Objects:
The basic building blocks that encapsulate both data (attributes) and behavior (methods). Objects represent real-world entities.

2. Classes:
Blueprints or templates from which objects are instantiated. A class defines the common properties and behaviors shared by all its objects.

3. Encapsulation:
The bundling of data and methods that operate on that data within a single unit (class), while restricting direct access to some components. This protects data integrity.

4. Inheritance:
A mechanism where a new class (subclass/derived class) acquires properties and behaviors from an existing class (superclass/base class), promoting code reuse and hierarchical classification.

5. Polymorphism:
The ability of different objects to respond to the same message (method call) in different ways. Achieved through method overriding and overloading, it increases flexibility.

6. Abstraction:
Hiding complex implementation details and exposing only essential features. Simplifies system modeling by focusing on what an object does rather than how it does it.

7. Message Passing:
Objects communicate by sending and receiving messages. This models real-world interactions between entities.

8. Modularity:
The system is decomposed into self-contained, loosely coupled objects/modules that can be developed, tested, and maintained independently.

Differentiation between Structured Methodologies and Object-Oriented Methodologies:
BasisStructured MethodologiesObject-Oriented Methodologies ApproachFunction-oriented; focuses on processes and functionsData-oriented; focuses on objects and data Basic UnitFunction/ProcedureObject (data + methods) DecompositionTop-down functional decompositionBottom-up composition of objects Data and FunctionsSeparated; global data shared among functionsEncapsulated together within objects ReusabilityLimited; functions are tightly coupled with dataHigh; classes and objects can be reused via inheritance and composition MaintenanceDifficult; changes in data structure affect many functionsEasier; changes localized within objects ModelingUses DFDs, Structure Charts, ERDs, Data DictionariesUses UML diagrams (use case, class, sequence, activity, etc.) System ViewProcess-centricEntity-centric Handling ComplexityBecomes difficult for large, complex systemsScales well for complex, evolving systems **Change**

AdaptabilityLess adaptable to requirement changesMore adaptable due to modularity and encapsulation ExamplesStructured Systems Analysis and Design Method (SSADM), Waterfall with structured analysisUnified Process (UP), Rational Unified Process (RUP), Agile with UML

Summary:

Structured methodologies were dominant in the 1970s–1980s and work well for smaller, well-defined systems with stable requirements. Object-oriented methodologies emerged to handle the complexity of modern software by modeling systems around real-world entities, providing better reusability, maintainability, and scalability. Today, OOAD is the standard for most large-scale software development, often combined with agile practices.

Note: These answers are prepared for educational purposes. Students are encouraged to verify with official textbooks and course materials.